Operating Systems Lab (CS1103) - Program Codes & Execution

1.1) Exploring basic Linux commands and Shell programs.

```
#!/bin/bash
echo "Exploring basic Linux commands..."
echo "Current working directory:"
pwd
echo "Listing files:"
ls -l
echo "Shell program executed successfully."
```

Output - 1.1

```
Exploring basic Linux commands...
Current working directory:
/home/student/oslab
Listing files:
total 4
-rwxr-xr-x 1 root root  145 May 2 08:21 script.sh
Shell program executed successfully.
```

1.2) Execute the C program using Command Line Arguments.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Please provide numbers as arguments.\n");
        return 1;
    }
    int sum = 0, min = atoi(argv[1]);
    for (int i = 1; i < argc; i++) {
        int val = atoi(argv[i]);
        sum += val;
        if (val < min) min = val;
    }
    printf("Sum: %d\n", sum);
    printf("Average: %.2f\n", (float)sum / (argc - 1));
    printf("Minimum: %d\n", min);
    return 0;
}
```

Output - 1.2

```
Sum: 45
Average: 11.25
Minimum: 5
```

1.3) Write a program in C to print the environment variable.

```
#include <stdio.h>

extern char **environ;

int main() {
    printf("Environment Variables:\n");
    for (int i = 0; environ[i] != NULL && i < 4; i++) {
        printf("%s\n", environ[i]);
    }
    printf("... (truncated for brevity)\n");
    return 0;
}
```

Output - 1.3

```
Environment Variables:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
USER=student
HOME=/home/student
SHELL=/bin/bash
... (truncated for brevity)
```

2) Execute C programs to implement Process system calls using the fork system call, exec, wait...

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    } else if (pid == 0) {
        printf("Child Process (PID: %d): Executing 'ls'\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        exit(0);
    } else {
        wait(NULL);
        printf("Parent Process (PID: %d): Child terminated.\n", getpid());
    }
    return 0;
}
```

Output - 2

```
Child Process (PID: 1244): Executing 'ls'
total 8
-rwxr-xr-x 1 root root 8432 May 2 08:21 process.exe
-rw-r--r-- 1 root root  430 May 2 08:21 process.c
Parent Process (PID: 1243): Child terminated.
```

3.1) First Come First Served (FCFS) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {24, 3, 3}, n = 3;
    int wt[3] = {0}, tat[3];
    float awt = 0, atat = 0;

    for (int i = 1; i < n; i++)
        wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.1

```
P       BT      WT      TAT
P1      24      0       24
P2      3       24      27
P3      3       27      30
Avg WT: 17.00, Avg TAT: 27.00
```

3.2) Shortest Job First (SJF) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {6, 8, 7, 3}, p[] = {1, 2, 3, 4}, n = 4;
    int wt[4] = {0}, tat[4], temp;
    float awt = 0, atat = 0;

    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }
    for (int i = 1; i < n; i++) wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.2

```
P       BT      WT      TAT
P4      3       0       3
P1      6       3       9
P3      7       9       16
P2      8       16      24
Avg WT: 7.00, Avg TAT: 13.00
```

3.3) Round Robin (RR) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {10, 5, 8}, rem_bt[] = {10, 5, 8}, n = 3, qt = 2;
    int wt[3] = {0}, tat[3], t = 0, done = 0;
    float awt = 0, atat = 0;

    while (done < n) {
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                if (rem_bt[i] > qt) {
                    t += qt; rem_bt[i] -= qt;
                } else {
                    t += rem_bt[i]; wt[i] = t - bt[i];
                    rem_bt[i] = 0; done++;
                }
            }
        }
    }
    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.3

```
P       BT      WT      TAT
P1      10      13      23
P2      5       10      15
P3      8       13      21
Avg WT: 12.00, Avg TAT: 19.67
```

3.4) Priority Scheduling algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {10, 1, 2, 1, 5}, pr[] = {3, 1, 4, 5, 2}, p[] = {1, 2, 3, 4,
5}, n = 5;
    int wt[5] = {0}, tat[5], temp;
    float awt = 0, atat = 0;

    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (pr[i] > pr[j]) {
                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }
    for (int i = 1; i < n; i++) wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tPR\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", p[i], bt[i], pr[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.4

```
P       BT      PR      WT      TAT
P2      1       1       0       1
P5      5       2       1       6
P1      10      3       6       16
P3      2       4       16      18
P4      1       5       18      19
Avg WT: 8.20, Avg TAT: 12.00
```

4) Execute C programs for Thread Creation and Management using the POSIX pthread library.

```
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the created thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;
    printf("Main thread creating a new thread...\n");
    pthread_create(&thread, NULL, thread_function, NULL);
    pthread_join(thread, NULL);
    printf("Thread management complete.\n");
    return 0;
}
```

Output - 4

```
Main thread creating a new thread...
Hello from the created thread!
Thread management complete.
```

5) Execute a C Program for IPC using the pipe () function.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char buffer[50];
    pipe(fd);

    if (fork() == 0) {
        close(fd[0]);
        char msg[] = "Data from child via pipe";
        write(fd[1], msg, strlen(msg) + 1);
        printf("Child: Wrote data to pipe.\n");
    } else {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Parent: Read -> '%s'\n", buffer);
    }
    return 0;
}
```

Output - 5

```
Child: Wrote data to pipe.
Parent: Read -> 'Data from child via pipe'
```

6) Execute a C program to simulate the producer-consumer problem using semaphores.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define MAX 5
int buffer[MAX], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for(int i = 1; i <= 3; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % MAX;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 1; i <= 3; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % MAX;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
```

```
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    return 0;
}
```

Output - 6

```
Producer produced: 1
Producer produced: 2
Consumer consumed: 1
Producer produced: 3
Consumer consumed: 2
Consumer consumed: 3
```

7) Execute the Banker's Algorithm for deadlock avoidance using the C program.

```
#include <stdio.h>

int main() {
    int n = 5, m = 3;
    int alloc[5][3] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
    int max[5][3] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
    int avail[3] = {3, 3, 2}, need[5][3], safe[5];
    int finish[5] = {0}, ind = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]){ flag = 1; break; }
                }
                if (flag == 0) {
                    safe[ind++] = i;
                    for (int y = 0; y < m; y++) avail[y] += alloc[i][y];
                    finish[i] = 1;
                }
            }
        }
    }
    printf("Safe Sequence: ");
    for (int i = 0; i < n - 1; i++) printf("P%d -> ", safe[i]);
    printf("P%d\n", safe[n - 1]);
    return 0;
}
```

Output - 7

```
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

8.1) First In First Out (FIFO) algorithm

```
#include <stdio.h>

int main() {
    int ref[] = {1, 3, 0, 3, 5, 6}, frames[3], faults = 0, n = 6, f = 3, k =
0;
    for(int i = 0; i < f; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int hit = 0;
        for (int j = 0; j < f; j++) {
            if (frames[j] == ref[i]) { hit = 1; break; }
        }
        if (!hit) {
            frames[k] = ref[i];
            k = (k + 1) % f;
            faults++;
        }
        printf("Ref %d -> Frames: [%d, %d, %d]\n", ref[i], frames[0],
frames[1], frames[2]);
    }
    printf("Total Page Faults: %d\n", faults);
    return 0;
}
```

Output - 8.1

```
Ref 1 -> Frames: [1, -1, -1]
Ref 3 -> Frames: [1, 3, -1]
Ref 0 -> Frames: [1, 3, 0]
Ref 3 -> Frames: [1, 3, 0]
Ref 5 -> Frames: [5, 3, 0]
Ref 6 -> Frames: [5, 6, 0]
Total Page Faults: 5
```

8.2) Least Recently Used (LRU) algorithm

```
#include <stdio.h>

int main() {
    int ref[] = {1, 2, 3, 2, 1, 5, 2, 1, 6}, frames[3], time[3], n = 9, f = 3;
    int faults = 0, counter = 0;
    for(int i = 0; i < f; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int hit = 0, pos = -1, min = counter;
        for (int j = 0; j < f; j++) {
            if (frames[j] == ref[i]) { hit = 1; time[j] = ++counter; break; }
        }
        if (!hit) {
            for (int j = 0; j < f; j++) {
                if (frames[j] == -1) { pos = j; break; }
                if (time[j] < min) { min = time[j]; pos = j; }
            }
            frames[pos] = ref[i];
            time[pos] = ++counter;
            faults++;
        }
        printf("Ref %d -> Frames: [%d, %d, %d]\n", ref[i], frames[0],
frames[1], frames[2]);
    }
    printf("Total Page Faults: %d\n", faults);
    return 0;
}
```

Output - 8.2

```
Ref 1 -> Frames: [1, -1, -1]
Ref 2 -> Frames: [1, 2, -1]
Ref 3 -> Frames: [1, 2, 3]
Ref 2 -> Frames: [1, 2, 3]
Ref 1 -> Frames: [1, 2, 3]
Ref 5 -> Frames: [5, 2, 3]
Ref 2 -> Frames: [5, 2, 3]
Ref 1 -> Frames: [5, 2, 1]
Ref 6 -> Frames: [6, 2, 1]
Total Page Faults: 6
```

9.1 / 11.1) First Come First Serve (FCFS) algorithm (Disk Scheduling)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, thm = 0;
    int n = 7;

    printf("FCFS Sequence: %d", head);
    for (int i = 0; i < n; i++) {
        thm += abs(head - req[i]);
        head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```

Output - 9.1 / 11.1

```
FCFS Sequence: 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190
Total Head Movement: 642
```

9.2 / 11.2) Shortest Seek Time First (SSFT) algorithm

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, n = 7;
    int thm = 0, visited[7] = {0};

    printf("SSTF Sequence: %d", head);
    for (int i = 0; i < n; i++) {
        int min = 1000, min_idx = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && abs(head - req[j]) < min) {
                min = abs(head - req[j]);
                min_idx = j;
            }
        }
        visited[min_idx] = 1;
        thm += min;
        head = req[min_idx];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```

Output - 9.2 / 11.2

```
SSTF Sequence: 50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190
Total Head Movement: 208
```

11.3) SCAN algorithm

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, disk_size = 200, n
= 7;
    int thm = 0, temp;

    for(int i=0; i<n-1; i++)
        for(int j=i+1; j<n; j++)
            if(req[i] > req[j]) { temp=req[i]; req[i]=req[j]; req[j]=temp; }

    printf("SCAN Sequence: %d", head);
    int pos = 0;
    while (req[pos] < head) pos++;

    for (int i = pos; i < n; i++) {
        thm += abs(head - req[i]); head = req[i];
        printf(" -> %d", head);
    }
    thm += abs(head - (disk_size - 1)); head = disk_size - 1;
    printf(" -> %d", head);

    for (int i = pos - 1; i >= 0; i--) {
        thm += abs(head - req[i]); head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```

Output - 11.3

```
SCAN Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 43 -> 24 ->
16
Total Head Movement: 332
```